Script Assembler – Generation of Test Scripts for KeTop Devices

DIPLOMA THESIS

submitted for the

Reife- und Diplomprüfung

at the

Department of Informatik

Submitted by:
Filip Schauer
Axel Csomány
Adam Höllerl

Supervisor:
Adrian Kern

Project Partner:
Martin Wieser, KEBA Group AG

Leonding, April 7, 2026

I hereby declare that I have composed the presented paper independently and on my own, without any other resources than the ones indicated. All thoughts taken directly or indirectly from external sources are properly denoted as such.

This paper has neither been previously submitted to another authority nor has it been published yet.

The presented paper is identical to the electronically transmitted document.

Leonding, April 7, 2026 Filip Schauer , Axel Csomány & Adam Höllerl

Abstract

Brief summary of our amazing work. In English. This is the only time we have to include a picture within the text. The picture should somehow represent your thesis. This is untypical for scientific work but required by the powers that are. Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.

Zusammenfassung

Zusammenfassung unserer genialen Arbeit. Auf Deutsch. Das ist das einzige Mal, dass eine Grafik in den Textfluss eingebunden wird. Die gewählte Grafik soll irgendwie eure Arbeit repräsentieren. Das ist ungewöhnlich für eine wissenschaftliche Arbeit aber eine Anforderung der Obrigkeit. *Bitte auf keinen Fall mit der Zusammenfassung verwechseln, die den Abschluss der Arbeit bildet!* Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.

Contents

Part I.

Introduction [X, F]

1. Background [X]

1.1. KEBA Group AG

For more than 50 years the technology company KEBA Group AG has been providing various customers with solutions to their unique problems. Among its three main branches, the Industrial Automation branch focuses primarily on the needs of the individual situation as the hardware and its software get built.

The company was founded 1968 in Linz, Austria. Today with 28 establishments in 16 different countries and more than 2000 employees they are internationally active.

2. Initial Problem [X]

2.1. The Problem

So called "KeTops" are used as operating devices to control machines of varying structure and complexity. These machines and their corresponding KeTops differ in their configuration from each customer which is why fitting test scripts are required. These test scripts examine all aspects of the device from the installed operating system to software specific behaviors.

The "Script Assembler" which is used in current production is able to merge a number of test scripts together for this use. The hard-coded nature and resulting limited maintainibility of the Script Assembler is its primary flaw. Furthermore all data concerning tests are saved and maintained in a TestRail database which although functional heavily limits the expandability and independence of the testing environment.

2.2. The Solution

This project aims to eliminate all of the listed problems and concerns while building upon and improving the existing capabilites of the Script Assembler.

- **EBNF parsing:**
  The existing structure of the test scripts should be formed into its own EBNF language which is then integrated into the Script Assembler. This eliminates a majority of the hard-coded parts of the software. Furthermore the editor should provide appropriate syntax highlighting when writing or editing a test script.

- **Keyword support:**
  New functionalities should be implemented which allows the editor of test scripts

to specify aspects like the version and the operating system the test has to be run on. The structure of keywords need to be precisely set in the EBNF language.

- **Database mock:**
  The structure of the currently used TestRail database should be studied and an accurate mockup should be created. A seamless switch between the mock and the real TestRail instance has to be implemented as well.

3. Used Technologies [F]

Just like the TRP project for which this project aims to provite an extension for, it was mainly written in the Python programming language. Aside from that, other tools have also been used or are otherwise important to be mentioned. The goal of this section is to give a broad overview of each of the technologies that we utilized or otherwise worked with when having written our project.

3.0.1. Python

Python is a high-level, interpreted, dynamically typed object oriented programming language with a focus on allowing programmers to create applications, scripts, or connections between existing programs, quickly.[1]

Part II.

EBNF [A, F]

4. Overview of EBNF [A]

4.1. Motivation for Formal Syntax Descriptions

To be able to run tests automated on every kind of KeTop, there is a need to modify test scripts depending on the given environment. That is why an extended Robot Framework language was needed. The Extended Backus-Naur Form (EBNF) was chosen for this, to make this new language easily extendable and deterministic to implement if adopted by other developers.

In practice, in the field of computer science, it is clear that a precise description of a language is the most important foundation. It is therefore essential that applications, communication protocols or programs function according to clear syntactic rules so that they are always interpreted in the same way by humans and machines. Natural language provides many different ways of describing something. There are ambiguities in language, and these can lead to errors in interpretation. These findings are not new. They have been known for a long time. Nevertheless, there is still no common standard that everyone adheres to, especially when languages have been implemented independently by different developers (cf. [2, p. V]).

Syntactic metalanguages exist to solve this problem. The goal of a metalanguage is to describe another language. This means there is a difference between the language that is described and the language used to describe it.

In programming, a metalanguage is a language used to describe the syntax and structure of another language, called the object language. For example, BNF and EBNF are metalanguages used to define programming languages such as ALGOL. The structure of sentences and expressions in a metalanguage can be defined using a so-called metasyntax. For example, one could indicate that the word "noun" can itself function as a noun in a sentence by writing: "noun" is a `<noun>`. (cf. [3])

4.2. The Backus-Naur Form (BNF)

One of the most important metalanguages for programming has been BNF (the so-called Backus-Naur Form). BNF started with ALGOL 60 in 1960. ALGOL 60 needed a clear, formal way to describe its syntax. John Backus and Peter Naur therefore created BNF. It made it possible to define language rules precisely. This was the first time syntax was described formally. The history of ALGOL 60 is crucial because it gave BNF its first real use. BNF became the foundation for defining many programming languages. It is still essential in compilers and language design today. (cf. [4, p. 3])

As BNF was first used for ALGOL 60 and became the model for many later languages, over time, people added changes or extensions, which created some confusion. Despite this, BNF showed the importance of clear, formal syntax.

Therefore, Extended BNF was later developed from BNF and included the most common extensions. The large amount of variants motivated the standardization effort that resulted in ISO/IEC 14977. (cf. [2, p. V])

4.3. The Evolution to Extended BNF (EBNF)

For early programming languages such as ALGOL 60, BNF was adequate. However, the increased complexity of modern languages required a more systematic notation to describe their syntax effectively.

Therefore, the so-called Extended Backus-Naur Form (EBNF) was introduced. The inventor of Extended BNF - EBNF, is Niklaus Wirth, a Swiss computer scientist. EBNF is based on BNF but it adds more elements.

> "One cannot fail to notice the lack of 'common denominators."

Niklaus Wirth stated this in his first article in the journal "Communications of the ACM" in 1977 about EBNF. [5]

He explained that there is a lack of common standards or unified fundamental principles across different language definitions and notational forms. Each language definition tends to use slightly different variants of the Backus-Naur form based notation. There is no generally accepted standard, and the differences are often small but unnecessarily varied. He stated that a more uniform notation would improve scientific work and comparison. EBNF is his solution for this problem.

As mentioned EBNF is clearly based on BNF, but it adds more elements. EBNF can define exact numbers of elements, describe exceptions, add comments, use flexible rule names, and allow extensions. This makes grammar definitions shorter and it is easier to read them. But EBNF also has restrictions. It can only handle languages with elements in order and is not able to describe very complex grammar. (cf. [2, pp. VII-VIII])

The main disadvantage of BNF according to ISO/IEC 14977 was:

> "Backus-Naur Form (used in ALGOL 60) has problems if the metasymbols `<` `>` `::=` occur in the language being defined. Some common forms of construction (e.g. comments) cannot be expressed naturally; other constructions (e.g. repetition) are long winded." [2, p. VII]

The added advantages, but also the limitations of EBNF, will be discussed in more detail in later chapters.

4.4. Wirth's Approach to Syntax Definition

Having established the historical need for a unified standard in the previous section, it is now necessary to examine the concrete syntactic rules proposed by Niklaus Wirth in 1977. In his article published in the journal Communications of the ACM (CACM), Wirth addressed the existing problems of syntactic notation directly. His contribution marked an important step in the formal development of language definitions.

Wirth did not intend to introduce a completely new theoretical concept. Instead, his objective was to improve the existing Backus-Naur Form in a systematic way. BNF had already proven to be useful. However, it contained ambiguities and practical limitations.

The notation presented in his article was therefore designed to be more practical and clearer in structure. It focused on improving readability and standardization rather than expanding theoretical expressive power. The goal was not innovation for its own sake. The goal was clarity, consistency, and easier application in real technical environments.

Wirth listed the following properties to extend BNF:

1. The notation clearly separates meta symbols, terminal symbols, and nonterminal symbols.
   Example: `syntax = "production".`
   Here, `syntax` is a nonterminal symbol. `"production"` is a terminal symbol. The equals sign and the dot are structural meta symbols.

2. Symbols used for structure can also be used as normal language symbols. This avoids artificial restrictions.
   Example: If quotation marks are used as meta symbols, they can still appear inside text if written twice, like `""` in many programming languages.

3. The notation contains a simple way to express repetition. This reduces the need for recursion in simple cases.

4. The notation avoids explicit representation of the empty string, such as by using `<empty>` or ε.

5. The notation is based on the ASCII character set. This makes it easy to use in most technical environments.

(cf. [5])

In his article, he showed the power of EBNF by defining its own syntax as an example (see figure 1):

> "This meta language can therefore conveniently be used to define its own syntax, which may serve here as an example of its use. The word identifier is used to denote nonterminal symbol, and literal stands for terminal symbol. For brevity, identifier and character are not defined in further detail." [5]

```
syntax      = {production}.
production  = identifier "=" expression ".".
expression  = term {"|" term}.
term        = factor {factor}.
factor      = identifier | literal | "(" expression ")" |
              "[" expression "]" | "{" expression "}".
literal     = """" character {character} """".
```

Figure 1.: Definition of EBNF in EBNF in "What Can We Do about the Unnecessary Diversity of Notation for Syntactic Definitions?" from N. Wirth [5]

4.5. Core Syntactic Extensions

The following overview focuses more in detail on the differences in EBNF and of the Extended Backus-Naur Form (EBNF) as described in Concepts of Programming Languages by Robert W. Sebesta. (cf. [6, pp. 149-151])

It is essential to note that these extensions do not enhance the descriptive power of BNF. The extensions are made to increase the readability and writability of formal grammars. BNF is theoretically sufficient for describing context-free languages, but EBNF provides a more concise notation for complex structures.

4.5.1. Core Syntactic Extensions in EBNF in detail

Sebesta identifies three primary extensions common to most versions of EBNF:

- **Optionality** (`[...]`):
  Brackets are used to denote a part of a right-hand side (RHS) that may occur once or not at all.
  Example: A C if-else statement can be described as: `<if_stmt>` → `if (<logic_expr>) <stmt> [else <stmt>]`.
  Instead one uses square brackets to make something optional. For example `["a"]` means “a” or empty.

- **Repetition** (`{...}`):
  Braces indicate that the enclosed part can be repeated any number of times, including zero. This replaces the need for recursion when describing simple lists.
  Example: A list of identifiers separated by commas: `<ident_list>` → `<identifier> {, <identifier>}`.
  Example: `{a}` means a, aa, aaa, and so on.

- **Multiple-Choice Options** (`(...)` and `|`):
  When one element must be selected from a group, the options are grouped in parentheses and separated by the OR operator.
  Example: Defining a term with different operators: `<term>` → `<term> (* | / | %) <factor>`.

In this notation, brackets, braces, and parentheses serve as metasymbols (notational tools) rather than terminal symbols of the language being described. (cf. [6, pp. 149-151])

4.5.2. Further Syntactic Extensions in EBNF based on ISO/IEC 14977

However, there are more differences between BNF and EBNF in the ISO/IEC 14977 definition: (cf. [2, p. VII])

- **Defining an explicit number of items:** In some cases it is needed to describe, that e.g., a field must have an exact length. In BNF, this is difficult. Often, the programmer has explained it then informally in English. Now EBNF solves this by allowing an integer followed by a repetition-symbol (`*`). With this solution, the programmer can specify the exact number of items. This makes rules much shorter.
- **Exceptions:** Sometimes it would be easier to define a rule by specifying what is not allowed. But BNF cannot easily say "any character except a quote." EBNF has a solution for that. EBNF uses an except-symbol (`-`) to define rules by exclusion. This makes definitions shorter and easier. Therefore, for example, a programmer can define a group of symbols and then subtract specific exceptional cases, such as "any character except a quote".
- **Formal comments:** Comments are an important feature, that BNF is not offering. EBNF provides a formal way to add comments using `(*` and `*)`. And these notes do not change the machine's logic.
- **Flexible names:** In BNF, names for categories often need brackets. EBNF uses an explicit comma (`,`) as a so-called concatenate symbol to link parts of a rule. This allows category names to use multiple words without brackets, meta-identifiers no longer have to consist of a single word or be enclosed in angle brackets. It also ensures that the layout of the syntax like spaces or line breaks has no effect on the defined language.
- **Extensions:** Programmers can add their own features to EBNF. These are called "special sequences" and are placed between question marks (`?`). This makes the start and end of any extension very easy to find. Extensions were used several

times in earlier versions of the solution presented in this thesis, but were eliminated in the final version because there was no longer a need for them.

4.5.3. Comparative Notation BNF and EBNF

While classical BNF is theoretically sufficient for describing any context-free language, it is often complicated and repetitive in practice. The main cause therefore is, that it relies, as described in the previous chapter, exclusively on recursion to express lists and optional elements. In the following illustration one can see the advantage of EBNF introducing specialized meta symbols, such as braces, brackets, and parentheses. [6, p. 153]

BNF and EBNF Versions of an Expression Grammar

```
BNF:
   <expr> → <expr>  +  <term>
          | <expr>  -  <term>
          | <term>
   <term> → <term>  *  <factor>
          | <term>  /  <factor>
          | <factor>
   <factor> → <exp>  **  <factor>
              <exp>
   <exp> → (<expr>)
         | id
EBNF:
   <expr> → <term> {(+  |  -) <term>}
   <term> → <factor> {(*  |  /) <factor>}
   <factor> → <exp> { ** <exp>}
   <exp> → (<expr>)
         | id
```

Figure 2.: Comparison between BNF and EBNF by Sebesta [6, p. 153]

As demonstrated in the example in figure 2, the structural differences lead to a significant reduction in the number of rules required:

- **Repetition via Braces in this example:** In the BNF version of the expression grammar, an `<expr>` must be defined recursively to handle a sequence of additions or subtractions. EBNF replaces this recursion with braces `{...}`, which denote that the enclosed part can be repeated indefinitely or left out entirely.
- **Multiple-Choice Options in this Example:** The example uses parentheses `(...)` combined with the OR operator `|` to group choices. This for example allows the EBNF version to handle both addition and subtraction in a single line. In BNF it requires a more complex approach.

4.5.4. Modern Variations and Standardization

While an official standard exists (ISO/IEC 14977:1996), it is rarely used in practice. There have several variations of EBNF emerged in recent years, which deviate from the standardized syntax, such as using a colon instead of an equal sign or a dot instead of a semicolon. Also the project described in this thesis partially deviates from ISO standards due to using the EBNF dialect of the Lark Python library.

4.6. ISO/IEC 14977 Standards for Assembler Logic

While the previous chapters describe the advantages of EBNF in terms of readability and compactness, the ISO definition has an additional function. The ISO standard ISO/IEC 14977 also defines strict technical rules.

These are very important for error-free processing by an automated tool such as a script assembler, as they ensure that the grammar definition remains mathematically unambiguous.

4.6.1. Operator Precedence

For every EBNF grammar, the strictly defined hierarchy of meta-symbols is essential, as it prevents ambiguities in the rules. The order of the symbols is clearly defined in the ISO standard without any leeway and is specified as follows: (the strongest is the repetition symbol. This takes precedence, followed by the exclude symbol, and so on).

1. Repetition symbol (*): For specifying exact numbers of repetitions.
2. Except symbol (-): For defining sets by exclusion.
3. Concatenate symbol (,): For explicitly linking consecutive elements.
4. Definition separator symbol (|): For separating alternatives.
5. Defining symbol (=): Formally assigns an expression to a nonterminal.
6. Terminator symbol (;): To clearly mark the end of a production rule.

(cf. [2, p. 1])

4.6.2. Layout Neutrality Through Gap Separators

Ideally, a script assembler should be able to process scripts regardless of their visual formatting. The ISO standard is helpful here, as it defines so-called “gap separators” (such as spaces, tabs, or line breaks) (cf. [2, pp. 5]). These characters have no formal meaning outside of terminal strings. This has the significant advantage of allowing for flexible grammar design without affecting the assembler’s logic. Since gap separators and comments have no formal effect on the defined language, the script assembler can ignore these characters during analysis, which, for example, increases its robustness against different programming styles.

In summary, several advantages arise. The grammar definition remains mathematically consistent for automated tools. Furthermore, it is beneficial that the grammar can be made more readable and understandable for human readers through formatting and annotations.

4.6.3. Syntactic Exceptions

As mentioned earlier, the Except symbol (-) allows the assembler to define specific instruction sets. An example of this would be “all characters except a semicolon.” However, the ISO standard also imposes very clear technical limitations on this function, as otherwise paradoxes could arise:

> “If a syntactic exception is permitted to be an arbitrary syntactic factor, Extended BNF could define a wider class of languages than the context-free grammars, including attempts which lead to Russell-like paradoxes... Such a license is undesirable and the form of a syntactic exception is therefore restricted to cases that can be proved to be safe.” [2, p. 2]

A good explanation for the Russell Paradox, formulated by Bertrand Russell, is the so-called barber’s example: In a village, the barber shaves precisely all the men who don’t shave themselves—and only these men. The question of whether the barber shaves himself leads directly to a contradiction: If he shaves himself, he shouldn’t, by definition; if he doesn’t shave himself, he should. (cf. [7])

This example illustrates that such self-referential definitions can lead to logical inconsistencies in formal systems.

4.6.4. Alternative Representations for System Compatibility

To guarantee that a grammar functions on different systems, the ISO standard allows the substitution of certain special characters if they are missing. For example, the characters `(:` and `:)` can replace `{` and `}`, or a period (`.`) can replace a semicolon at the end of a rule. This ensures that the grammar remains readable and functional on different platforms without losing its original structure. (cf. [2, pp. 5])

4.6.5. Special Sequences as a Formal Interface for Assembler Extensions

Although special sequences were already described in the previous section as a general extension option, they are listed again here to specifically clarify their "technical" role as placeholders. In the logic of a script assembler, they serve as a standardized mechanism to mark parts of the language that cannot be defined by EBNF itself (e.g., specific character sets). (cf. [2, p. VII])

By enclosing them in question marks (`?...?`), the grammar remains formally valid for the assembler tool. In this work, they thus can provided the flexibility for possible future functional extensions without violating the fundamental syntactic rules of the ISO standard. (cf. [2, p. 3])

4.6.6. Evaluating the Standard for our Script Assembler

After introducing the theoretical foundations of EBNF, it is necessary to evaluate how the ISO standard was applied in the context of the script assembler. The goal of the project was not to implement a complete parser for the Robot Framework. Instead, the focus was on building a preprocessor for specific control tags such as `@OS`. For this reason, the resulting grammar is not a strict implementation of the ISO standard. It represents a pragmatic combination of formal rules and practical requirements.

A central aspect of this evaluation is the concept of layout neutrality. The ISO standard defines spaces, tabs, and line breaks as so-called gap separators. These elements have no formal meaning and can be ignored by the parser. This principle improves flexibility and readability. However, it could not be applied completely in this project. Inline spaces and tabs between control tokens are treated as insignificant and are therefore ignored. This allows test scripts to remain flexible in their formatting.

Line breaks, however, require a different treatment. Robot Framework scripts are structured line by line. This means that line breaks carry semantic meaning. They define the structure of the script. Therefore, line breaks could not be treated as layout-neutral elements. Instead, they were explicitly defined as structural tokens in the grammar. This is a necessary deviation from the ISO specification to ensure correct processing of the input.

Further deviations from the standard were made in the handling of syntactic constraints. The ISO standard introduces syntactic exceptions to define rules by exclusion. However, this mechanism can become complex and difficult to understand. In some cases, it can even lead to problematic definitions. For this reason, syntactic exceptions were not used in the assembler.

Instead, the implementation relies on regular expressions. Regex provides a practical way to capture and process text patterns. It allows the parser to treat sections of Robot Framework code as plain content. The internal structure of these sections is not analyzed. This simplifies the grammar and reduces its complexity. As a result, the grammar becomes easier to read and maintain.

5. Approaches to EBNF [A]

6. A Problem with EBNF [F]

7. Our EBNF [F]

Part III.

Implementation [A, F, X]

8. Preprocessor [A]

9. Parser [F, A]

10. Syntax Highlighting [A]

11. Extensibility [F]

12. Database [X]

Besides the extension of the Script Assembler's capabilities and the creation of a new EBNF language that makes the software more extendable, another main focus of the project is the creation of a mockup of the TestRail database. This mockup is not only made for testing purposes, but allows the user to work independently of the TestRail instance. The mockup is designed to be as close to the real TestRail database as possible, allowing a seamless switch between the two instances. This means that all functionalities that are available in the real TestRail database are also available in the new mockup.

This section aims to give an overview of the different types of databases, the structure and functionalities of the TestRail database specifically, the resulting design and implementation of the mockup database, and the way the database switch is finally implemented.

12.1. Types of Databases

Depending on the requirements of a given project, different types of databases are suitable for different use cases. It is important to analyze what qualities are needed and which qualities are negligible to find the best fitting solution.

The most important qualities of a database type can be expressed in the following categories:

- **Data Model:**
  The data model of a certain database type describes how the data is structured and stored. This can be in the form of tables, documents, graphs or other structures. The data model has a notable impact on aspects like performance and scalability.
- **Storage Architecture:**
  The storage architecture of a database type not only describes how the data is

stored, but also how it may be accessed, managed and distributed. This can be in the form of a single server, a cluster of servers or even a distributed network of servers across the globe.

- **Query Model:**
  The query model of a database type describes the means of accessing the data. This can occur in the form of a specifically designed query language, an API or other methods. This has an undeniable impact on the ease of use of the database and how the user writes and optimizes queries.

Based on these qualities, various databases types have been developed, each with their own unique advantages and disadvantages. Some of the most common types of databases include but are not limited to relational databases, object-oriented databases and NoSQL databases, which can be further categorized by the data model they use, such as document stores, key-value stores, column-family stores and graph databases. (cf. [8])

12.2. Relational Databases

For the purposes of this project, we will take a closer look at relational databases, as this is the type of database that TestRail uses.

12.2.1. Structure of Relational Databases

Relational databases are based on the relational model, which organizes its data into tables, consisting of rows and columns. Each table represents a specific entity and each row in the table represents an instance of that entity. The columns in the table represent the attributes of the entity.

Relationships between these entities can be established through the use of keys. A primary key is a unique identifier for each instance of an entity, while a foreign key is a reference to the primary key of another entity. Meaningful relationships between entities can be built with the usage of these keys, allowing complex queries to be executed across multiple linked entities.

12.2.2. ACID

ACID is an acronym that stands for the four key properties of a safe database transaction:

- **Atomicity:**
  Atomicity ensures that a transaction, which is often a sequence of multiple operations, is treated as a single unit. This means that either all operations within a given transaction are executed successfully, or none of them are. If any operation of the transaction fails due to an error, a power outage or any other issue, the entire transaction is rolled back to its previous state before the transaction began. This guarantees that the database remains consistent and prevents data corruption or loss that could occur if only part of the transaction were to be executed.
- **Consistency:**
  Consistency ensures that the database can only be brought from one valid consistent state to another, while following all defined rules, constraints and triggers. Invalid transactions that violate any of the defined rules are rolled back accordingly, ensuring that the database remains consistent.
- **Isolation:**
  Isolation ensures that concurrent transactions have no impact on each other. This means that the effects of a transaction in progress are not visible to other transactions until it completes successfully. This prevents a number of issues that would otherwise arise from concurrent transactions, like dirty reads, where one transaction reads data that has been modified by another transaction but not yet fully committed, potentially leading to incorrect results.
- **Durability:**
  Durability ensures that once a transaction has been successfully completed and committed, the state of the database persists even in the case of a crash, power failure or any other issue.

Databases that adhere to these properties are considered to be reliable and safe for use in critical environments, as they guarantee that the data not only remains consistent, but also that it is not lost or corrupted due to unforseen events. (cf. [9])

12.2.3. Advantages and Disadvantages of Relational Databases

One of the main strengths of relational databases is their ability to handle long and complex queries across multiple linked entities with the usage of foreign keys and joins. The consistency and integrity of the data is guaranteed by the ACID properties. On top of that, relational databases are some of the most widely used and well-known types of databases, which means that there are a lot of resources and a large community available for support and development.

However, relational databases can struggle with scalability and performance when dealing with large volumes of data or high traffic. The fixed schema of relational databases can also be a disadvantage, as it can make it difficult to adapt to changing requirements or to handle unstructured data. Additionally, the execution of complex queries can lead to performance issues on larger datasets.

12.3. The TestRail Database

TestRail is a web-based test management tool that provides a wide range of features for managing and organizing test cases, test runs and test results. The relational database in the background of TestRail is used to store all of the data related to these features. The user can interact with the database directly by using the TestRail API, although the database itself is not directly accessible to the user.

Thus, for the creation of the mockup database, the structure and functionalities of the TestRail database have to be studied and analyzed based on the API documentation and the structure of its response data. While the mockup database doesn't have to be an exact replica of the TestRail database, the functionalities used by the Script Assembler need to deliver responses with identical structure to ensure the switch between databases can occur without any issues.

In order to achieve this, the most important entities and their relationships have to be identified.

12.3.1. Test Cases

12.3.2. Test Runs

12.3.3. Test Results

12.3.4. Projects, Sections and Suites

12.3.5. Users

12.4. The Mockup Database

12.5. Switching Between Databases

The switch between the two databases is implemented in a way that allows the user to easily and definitively choose which database should be used at the execution of the python script. This is achieved by using a command line argument that can be passed when executing the script. The argument is then processed and the appropriate database instance is initialized based on if the argument is present or not.

```
python ./broker.py
```

Here, the standard TestRail database is used, as no argument is passed to the script.

```
python ./broker.py -db
# OR
python ./broker.py --database
```

Here, the PostgreSQL mockup database is used, as the argument is passed to the script. The argument can either be a short version, or a long verion, as both are processed in the same way and lead to the same result.

1

References

[1] Python Software Foundation, "What is Python? Executive Summary," retrieved 2026-02-21. [Online]. Available: https://www.python.org/doc/essays/blurb/

[2] ISO/IEC 14977, "Information technology - Syntactic metalanguage - Extended BNF," *International Standard 14977:1996(E)*, vol. 1996, pp. 1–24, 1996. [Online]. Available: http://www.iso.org/iso/catalogue_detail.htm?csnumber=26153

[3] Wikipedia contributors, "Metalanguage," 2026, accessed: 2026-02-23. [Online]. Available: https://en.wikipedia.org/wiki/Metalanguage

[4] H. Mössenböck, *Compiler Construction*, 2025.

[5] N. Wirth, "What Can We Do about the Unnecessary Diversity of Notation for Syntactic Definitions?" vol. 20, no. November, pp. 1–2, 1977.

[6] R. W. Sebesta, *Concepts of Programming Languages ELEVENTH edition*.

[7] Wikipedia contributors, "Barbier Paradoxon," 2026, accessed: 2026-02-23. [Online]. Available: https://de.wikipedia.org/wiki/Barbier-Paradoxon

[8] Vinayak Baranwal, "What are the different types of databases? explained with use cases and architectures," 2025, accessed: 2026-02-28. [Online]. Available: https://www.digitalocean.com/community/conceptual-articles/database-types

[9] Wikipedia contributors, "Acid," 2026, accessed: 2026-02-28. [Online]. Available: https://en.wikipedia.org/wiki/ACID

List of Figures

List of Tables

List of Listings

Appendix